THE NATIONAL UNIVERSITY OF LESOTHO
Department of Mathematics and Computer Science
Faculty of Science and Technology



CS4433: SOFTWARE ENGINEERING

Phase 3: System Design

Bakoena Thamae 202321319
BSc Information Systems

Table of Contents

1. Introduction....3
2. Architectural Design....4
2.2 Components and responsibilities....4
3. Detailed Class Design....5
3.1 Core class specifications....6
4. Design Patterns, SOLID and Security....7
5. Behavioural Models....8
6. Database Design....11
6.1 ERD and normalisation....11
6.2 Data dictionary....11
6.3 MySQL schema....13
6.5 Class-to-table mapping....14
7. Requirements-to-Design Traceability....14
8. Verification and Test Design....15
9. Design Decisions and Limitations....16
References....17
Appendix A: MySQL DDL....18
Appendix B: Diagram Tool Evidence....22
Appendix C: MySQL Workbench Tool Evidence....25

1. Introduction

This document presents Phase 3—System Design for the Moderated Learning Management System (LMS). It translates the agreed requirements into an implementable architecture, class structure, behavioural models, relational database schema and testable design decisions. The system supports Administrator, Lecturer, Head of Department (HoD) and Student roles, and is intended to operate offline on a single-school computer or local network using a C++17 console or simple desktop interface and a local MySQL database (National University of Lesotho, 2026).

The design uses UML to communicate static structure and system behaviour. It follows separation of concerns: user-interface code calls application services, application services enforce the rules, and repositories handle persistence. UML notation follows the OMG UML specification (Object Management Group, 2017).

- Enforce role-based access and keep student marks hidden until the HoD has moderated and released them.
- Support course enrolment, assignment creation, individual extensions, submissions and resubmissions, late penalties, marking and written feedback.
- Record every mark change with the acting user, timestamp, previous value, new value and reason.
- Store application data locally in MySQL and support offline operation.
- Keep business rules independently testable from the interface and SQL implementation.
- Support the expected school scale of approximately 800 students through indexed queries and performance testing.

Out of scope are online payment, video conferencing, automated grading, plagiarism detection, and web/mobile interfaces. The design does not assume that these features will be added during this phase.

Topic	Design position	Status
Technology	C++17 application, local MySQL 8.0+ database, draw.io UML diagrams and MySQL Workbench EER model.	Design baseline
Network	The system must operate without internet access; a local network may be used where needed.	Requirement
Late work	10% of the assignment maximum is deducted per late day for up to three days; beyond the window, the assignment outcome is zero or submission is rejected, depending on the stakeholder decision.	Needs confirmation: accept-with-zero vs reject
Late-day calculation	Use calendar days unless stakeholders approve working days; calculate from the effective deadline after an approved extension.	Proposed; confirm at Gate 2
Resubmission	Each permitted attempt is a new submission row with an increasing version number; historical attempts are retained.	Design decision
Mark visibility	Students can view marks and feedback only after the HoD releases the approved result.	Requirement
File storage	Files are stored under an application-controlled directory; the database stores	Design decision; file type/size limits need approval

	validated metadata and path.	

2. Architectural Design

2.1 Selected architecture: layered architecture

A layered architecture separates the user interface, application services, domain rules and data-access responsibilities. It suits the LMS because role permissions, deadlines, penalties, moderation and audit rules must be applied consistently regardless of which menu initiated an operation. It also reduces the amount of presentation and SQL code that must change when a business rule is updated (Sommerville, 2016).

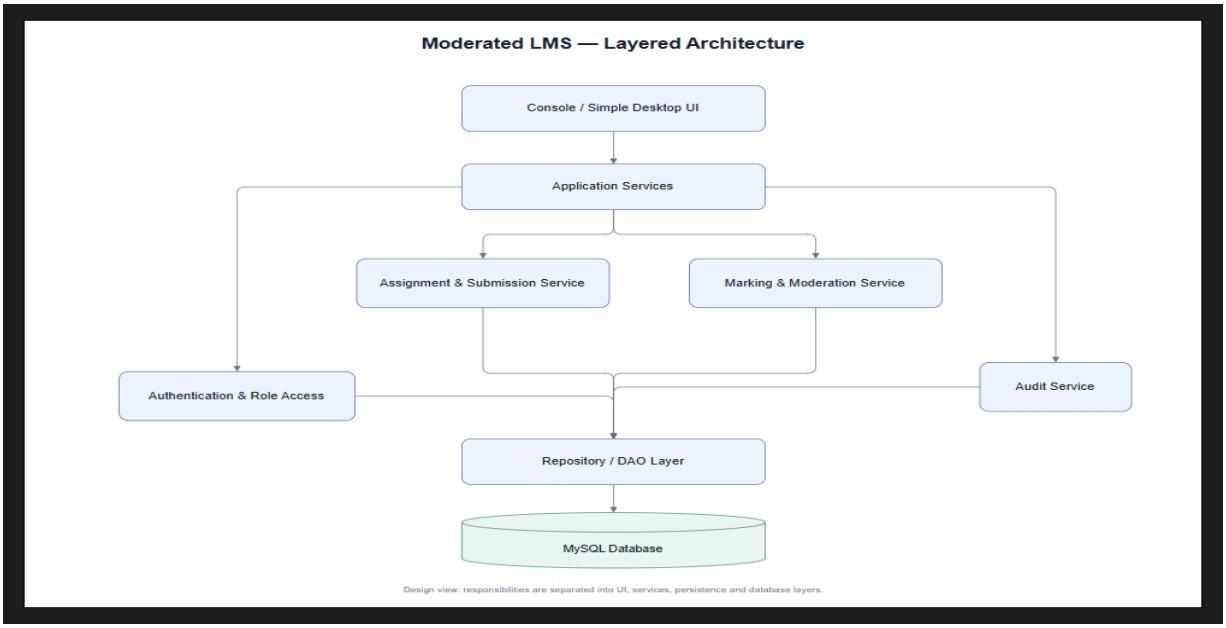


Figure 1. Layered architecture of the Moderated LMS (draw.io screenshot supplied for this assignment).

2.2 Components and responsibilities

Component	Responsibilities	Depends on / interacts with
Console / simple desktop UI	Collect input, display menus, show validation messages and results; no direct SQL.	Application services
Application services	Coordinate use cases, validate input, enforce role permissions and orchestrate transactions.	Domain policies, repositories, authentication context
Authentication and role access	Verify password hashes, establish user context and check role/course/department scope.	UserAccount repository; password-hashing library
Assignment and submission service	Create assignments, grant extensions, check effective deadlines, accept valid attempts and retain versions.	Deadline policy, SubmissionRepository, AssignmentRepository
Marking and moderation service	Record marks and feedback, return marks for correction, moderate and release approved results.	MarkRepository, SubmissionRepository, AuditService

Audit service	Write immutable audit entries for mark changes and release/moderation actions.	AuditRepository; transaction context
Repository / DAO layer	Execute parameterised SQL, map rows to domain objects and expose persistence interfaces.	MySQL connector and database
MySQL database	Persist users, courses, enrolments, assignments, extensions, submissions, marks and audit records.	Repositories

2.3 Alternatives considered and rejected

Model–View–Controller (MVC) was considered. MVC is useful where multiple views and complex presentation behaviour dominate, but this project uses a console or simple desktop interface and has a short delivery window. Layered architecture gives the required separation with less presentation-framework overhead. A monolithic design with SQL embedded in menu handlers was rejected because it would mix concerns and make permission checks, audit logging and unit testing harder.

2.4 Key architecture rules

- The UI calls application services; it does not execute SQL directly.
- Application services establish the authenticated actor and check permissions before protected reads or writes.
- Repositories handle persistence and mapping, not authorisation decisions.
- Mark changes and their audit records are committed in the same transaction.
- Every resubmission is retained as a separate version; a new attempt does not overwrite history.
- Passwords are stored as salted password hashes produced by a recognised password-hashing library; plaintext passwords are never stored.
- All SQL values are passed through parameterised/prepared statements.

3. Detailed Class Design

The class model represents the main domain concepts and their associations. The supplied class-diagram screenshot is retained as the diagram evidence; the following specifications clarify responsibilities and resolve the model-to-schema mapping.

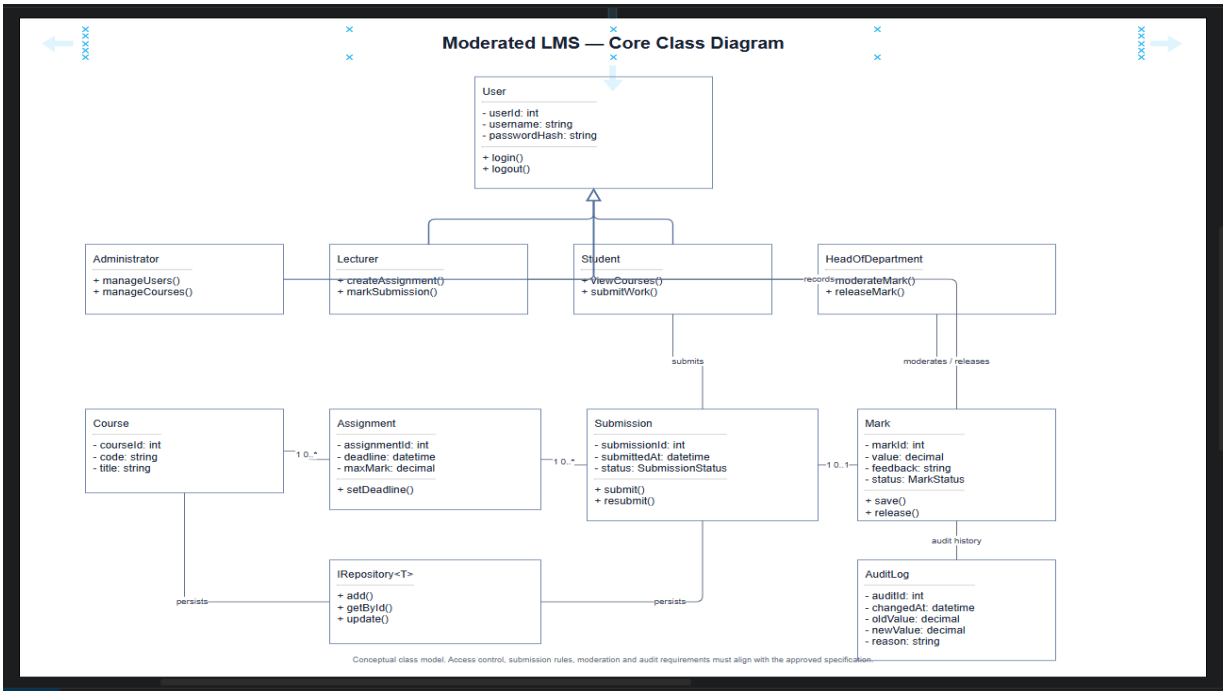


Figure 2. Core class diagram of the Moderated LMS (draw.io screenshot supplied for this assignment).

Alignment note: Figure 2 is the supplied core domain-class view. The ERD and data dictionary add persistence-support entities that are not separate boxes in this overview: Department, LecturerCourse and Enrolment represent organisational and many-to-many relationships, while ExtensionGrant records an individual deadline change. The diagram's Mark and AuditLog concepts map to mark_record and audit_log. Section 6.5 provides the complete mapping.

3.1 Core class specifications

Class / interface	Key attributes	Main operations / responsibility
User (abstract)	userId, username, passwordHash, fullName, email, active	Shared identity and authentication data; login() and logout().
Administrator	Inherited User identity	Manage users, departments, courses, lecturer assignments and student enrolments.
Lecturer	Inherited User identity	Create assignments, grant individual extensions for assigned courses, mark submissions and write feedback.
Student	Inherited User identity	View enrolled courses and assignments, submit/resubmit work, view released marks and feedback.
HeadOfDepartment	Inherited User identity and department scope	Review/moderate marks, return marks for correction, approve and release results.
Department	departmentId, name	Group courses and provide the scope for HoD moderation.
Course	courseId, courseCode, title, departmentId	Represent a school course and its department; link students and lecturers.
Assignment	assignmentId, courseId, lecturerId, deadline, maxMark	Define the work, deadline, maximum mark and instructions.
Submission	submissionId, assignmentId, studentId,	Represent a single attempt and its lifecycle

	submittedAt, fileLocation, status, versionNo	status.
MarkRecord	markId, submissionId, lecturerId, markValue, feedback, moderationStatus	Store the lecturer's mark and feedback plus moderation/release state.
AuditEntry / AuditLog	auditId, markId, actorId, actionType, changedAt, oldValue, newValue, reason	Keep an auditable history of mark and moderation changes.
SubmissionRepository / MarkRepository	Persistence interface methods	Hide SQL behind add/get/update operations; MySQL implementations use parameterised statements.
IPenaltyStrategy	Penalty calculation contract	Calculate a penalty from late days, maximum mark and the approved policy; StandardLatePenalty implements the current rule.

3.2 Relationships and multiplicities

- One Department has zero or many Courses; each Course belongs to exactly one Department.
- One Department may be associated with many user accounts; a user account may have no department (for example, a system-wide administrator) or one department.
- Lecturers and Courses have a many-to-many assignment relationship resolved by lecturer_course.
- Students and Courses have a many-to-many enrolment relationship resolved by enrolment; a student-course pair is unique.
- One Course has zero or many Assignments; each Assignment belongs to one Course and is created by a lecturer assigned to that course.
- One Assignment may have many Submission attempts; each attempt belongs to one student and one assignment. The combination of assignment_id, student_id and version_no is unique.
- A Submission has zero or one MarkRecord. A MarkRecord may have many AuditLog entries.
- Extensions are linked to one assignment and one student, record the granting user, revised deadline and reason; an assignment/student pair has at most one current extension in this baseline schema.

4. Design Patterns, SOLID and Security

4.1 Strategy pattern — late-penalty policy

IPenaltyStrategy defines a penalty-calculation contract, and StandardLatePenalty implements the approved school rule. The submission or marking service receives the policy through an interface rather than embedding the formula in a menu handler. This isolates a policy that may change and allows boundary tests. The provisional formula is: 0 late days = no penalty; 1–3 late days = 10% of the assignment maximum per late day; more than 3 late days = zero outcome or rejection, pending the stakeholder decision. An approved extension changes the effective deadline before lateness is calculated.

4.2 Repository / DAO pattern — persistence

SubmissionRepository, AssignmentRepository, MarkRepository and AuditRepository isolate SQL from business logic. MySQL implementations use prepared statements, foreign keys and transactions. Tests can substitute fake repositories for fast unit testing. Repositories do not decide whether the current actor is authorised; application services perform those checks.

Principle	Application in this design
Single Responsibility (SRP)	UI displays information; services coordinate use cases; penalty policy calculates deductions; repositories persist data; AuditService records audit events.
Open/Closed (OCP)	A new late-penalty policy can implement IPenaltyStrategy without rewriting submission menus or repository code.
Liskov Substitution (LSP)	Any IPenaltyStrategy implementation must honour the same input/output contract and boundary conditions.
Interface Segregation (ISP)	Use focused repository interfaces, such as SubmissionRepository and MarkRepository, rather than one oversized persistence interface.
Dependency Inversion (DIP)	Application services depend on interfaces/policies, not directly on concrete MySQL repository classes; concrete implementations are supplied during application setup.

4.4 Security and integrity decisions

- Authorise every protected operation in application services, not only by hiding menu choices.
- Students may access only their own submissions and released results; lecturers may act only on assigned courses; HoDs may moderate and release marks only within their department.
- Store password hashes, use parameterised SQL, validate inputs and restrict direct database access.
- Record actor, timestamp, old value, new value and reason for every mark change; record release and moderation events as well.
- Commit the mark update and audit entry in one transaction. If either write fails, roll back both.
- Validate that marks are not negative and do not exceed the assignment maximum. The cross-table maximum check is enforced in the application service within the same transaction, not by a simple single-table CHECK.
- Back up the local database regularly and test restoration; keep backups in a controlled location.

5. Behavioural Models

5.1 Submission state machine

The state model distinguishes Draft, Submitted, Late, Marked, Moderated, Correction Required, Released and End. The application validates each transition and persists it. A resubmission creates a new version and does not erase an earlier attempt. The permitted late window and the behaviour beyond three days must match the final approved requirements.

Schema alignment: SUBMITTED, LATE, MARKED, MODERATED and RELEASED are submission.status values. The Correction Required state represents mark_record.moderation_status = 'RETURNED'; it is a moderation outcome rather than a submission.status value. The application service coordinates the related state updates.

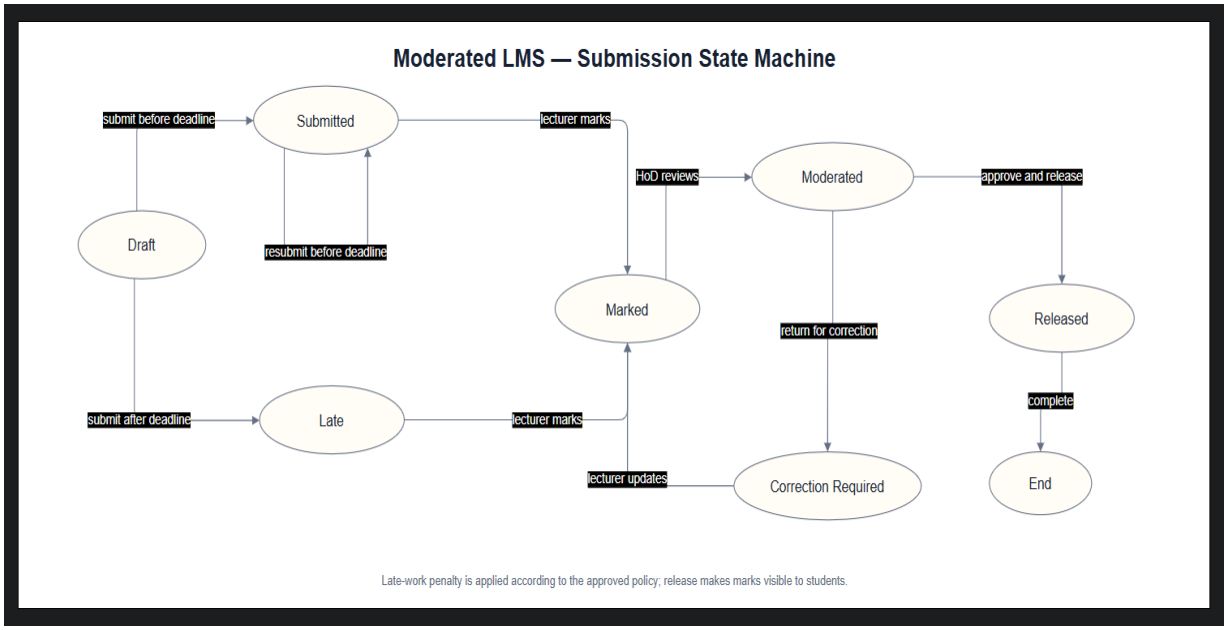


Figure 3. Submission state machine (draw.io screenshot supplied for this assignment).

5.2 End-to-end activity with swimlanes

The activity model shows the student preparing and submitting work, the LMS checking the deadline/extension and recording the timestamp, the lecturer marking and recording feedback, and the HoD moderating before release. Students must not see marks before the HoD has released them.

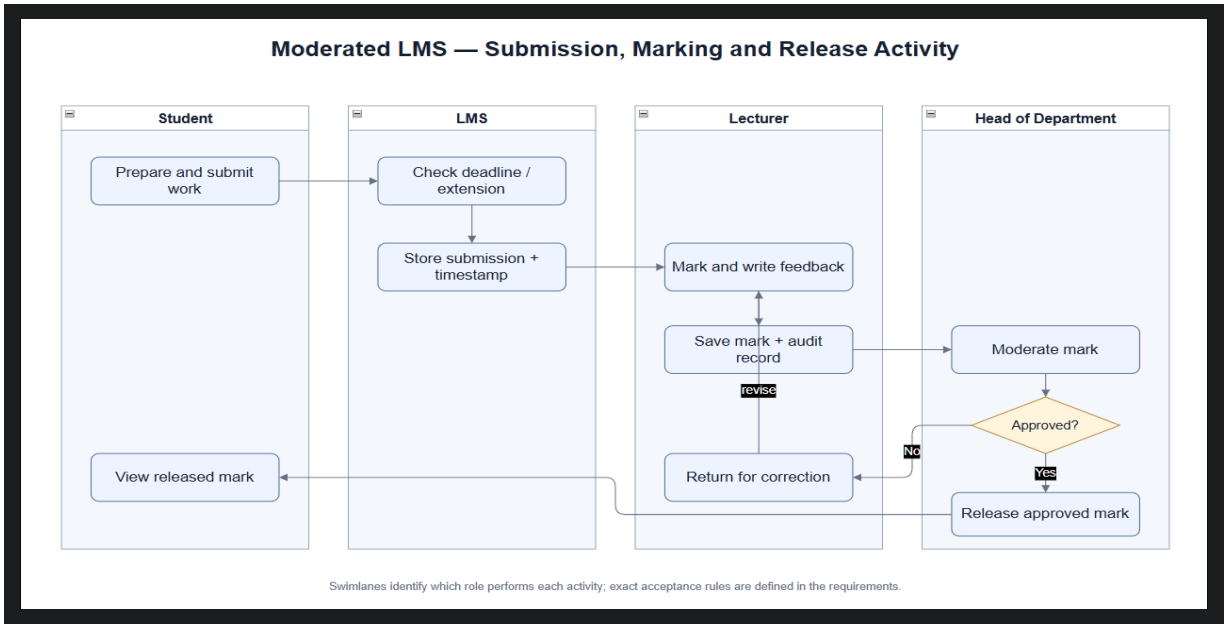


Figure 4. Submission, marking and release activity (draw.io screenshot supplied for this assignment).

5.3 Sequence diagram — submit assignment

The submission interaction verifies the student session and enrolment, obtains the effective deadline, checks the submission window, stores the file and metadata, assigns the next version number, and returns a confirmation. Invalid files, unauthorised access, duplicate versions and submissions outside

the approved window must produce clear errors. The database transaction must avoid saving partial metadata when a file or database operation fails.

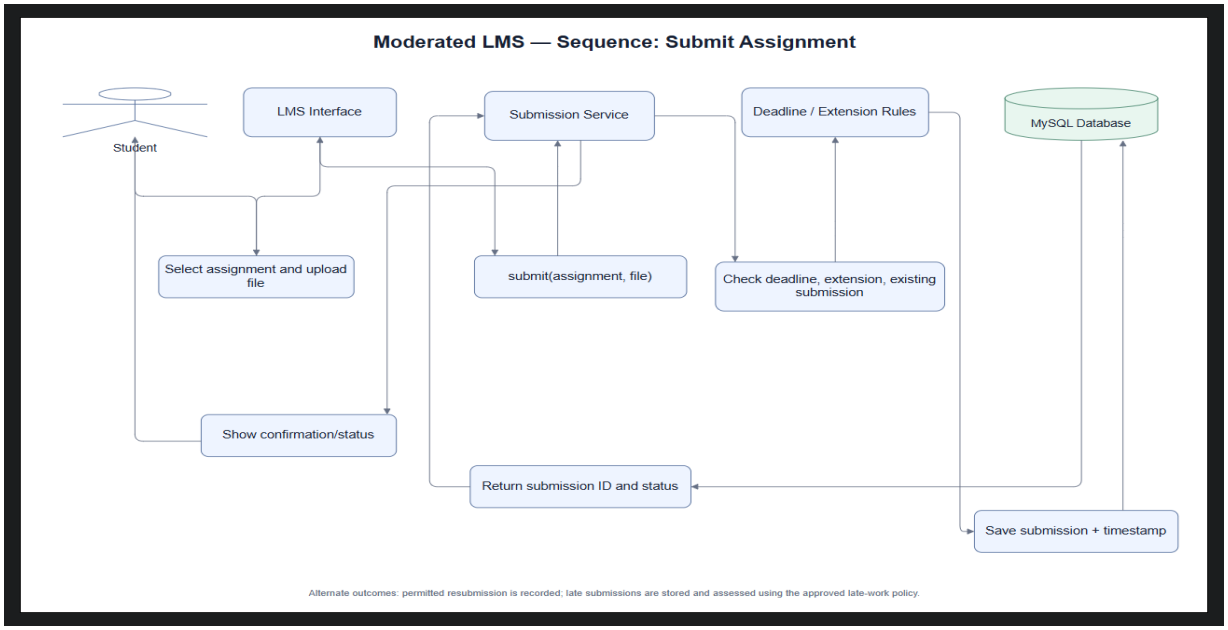


Figure 5. Assignment submission sequence (draw.io screenshot supplied for this assignment).

5.4 Sequence diagram — moderate and release marks

The lecturer records a mark and feedback; the system stores the mark and audit event. The HoD reviews the mark and may approve it or return it for correction. Release is a separate explicit action. Only approved and released results become visible to the student. Changes after release, if allowed, must be re-audited and require a defined policy.

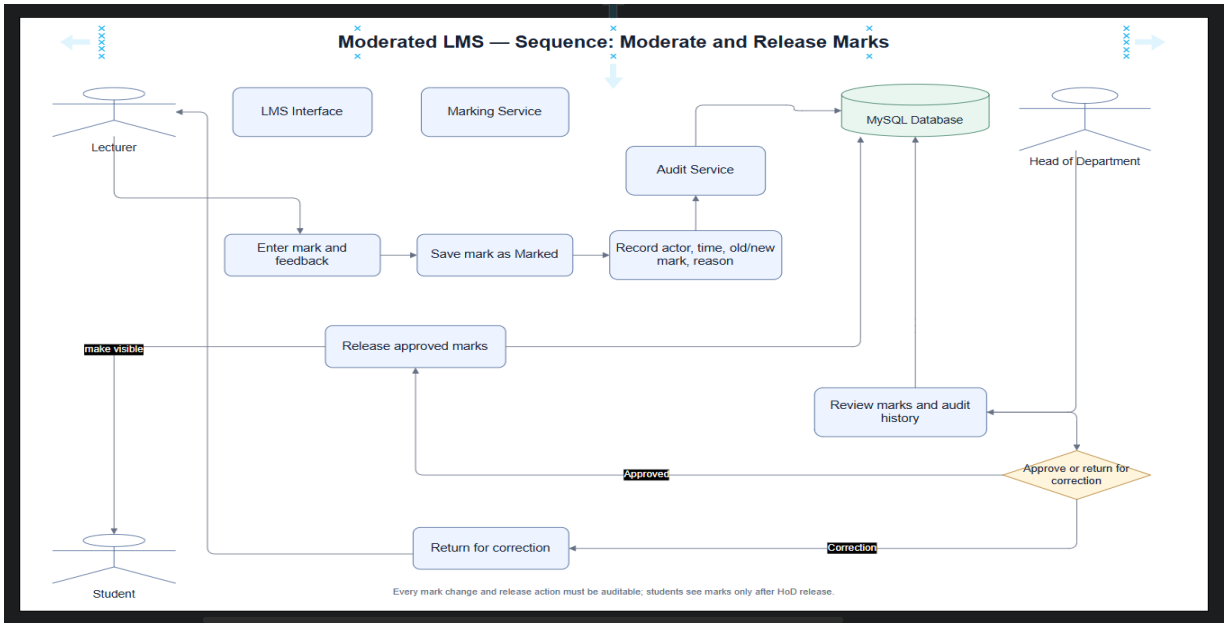


Figure 6. Mark moderation and release sequence (draw.io screenshot supplied for this assignment).

6. Database Design

6.1 ERD and normalisation

The ERD below is a logical view of the same entities, keys and relationships defined by the corrected MySQL DDL in Appendix A. It replaces the previously inserted Workbench screenshot, which showed a different timetable database and was not a valid LMS ERD. The DDL targets MySQL 8.0.16+ and uses primary keys, foreign keys, uniqueness rules and InnoDB. The schema is designed toward Third Normal Form: repeating groups are separated, many-to-many relationships are resolved through junction tables, and audit history is stored separately rather than overwritten.

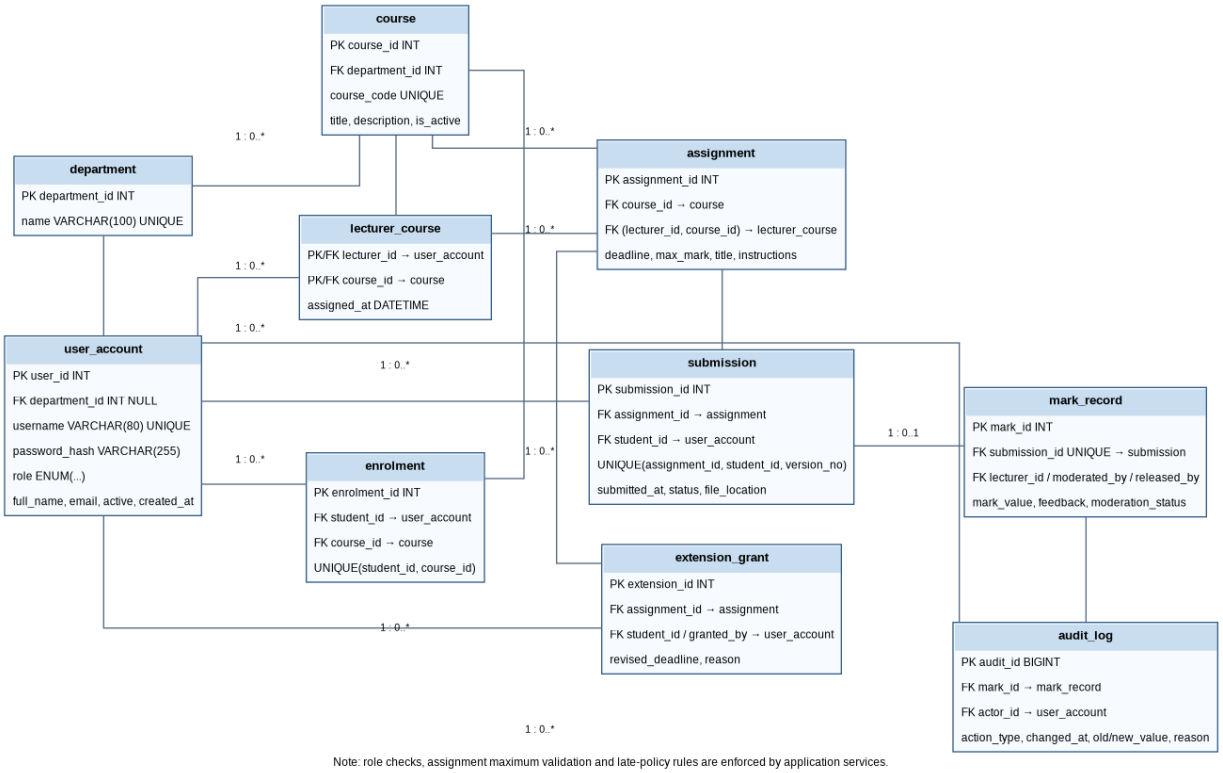


Figure 7. Moderated LMS logical ERD derived from the corrected MySQL schema. Generate the editable EER model in MySQL Workbench using the included DDL before final tool-evidence submission.

Tool evidence: the six UML screenshots above show the diagrams open in draw.io. The previously supplied MySQL Workbench screenshot demonstrates that Workbench was open, but it showed an unrelated schema and has therefore not been misrepresented as the LMS ERD. To provide accurate MySQL Workbench evidence, execute Appendix A in a development schema, choose Database > Reverse Engineer, select moderated_lms, arrange the EER diagram, save the .mwb model and capture a screenshot of that actual LMS model.

6.2 Data dictionary

Table	Field(s)	Type / constraint	Purpose
department	department_id	INT, PK, AUTO_INCREMENT	Unique department identifier.
department	name	VARCHAR(100), NOT NULL, UNIQUE	Department name; duplicates are prevented.

user_account	user_id	INT, PK, AUTO_INCREMENT	Unique account identifier.
user_account	department_id	INT, nullable FK → department	Optional department association; may be NULL for system-wide accounts.
user_account	username	VARCHAR(80), NOT NULL, UNIQUE	Login name.
user_account	password_hash	VARCHAR(255), NOT NULL	Salted password hash from a recognised password-hashing library; never plaintext.
user_account	full_name	VARCHAR(160), NOT NULL	Name displayed in the application.
user_account	email	VARCHAR(160), nullable, UNIQUE	Optional email address.
user_account	role	ENUM, NOT NULL	One of ADMINISTRATOR, LECTURER, HOD or STUDENT.
user_account	is_active	BOOLEAN, NOT NULL, default TRUE	Enables/disables an account without deleting history.
user_account	created_at	DATETIME, NOT NULL	Account creation timestamp.
course	course_id	INT, PK, AUTO_INCREMENT	Unique course identifier.
course	department_id	INT, NOT NULL, FK → department	Owning department.
course	course_code	VARCHAR(30), NOT NULL, UNIQUE	Unique course code.
course	title	VARCHAR(160), NOT NULL	Course title.
course	description	TEXT, nullable	Optional course description.
course	is_active	BOOLEAN, NOT NULL, default TRUE	Indicates whether the course is active.
lecturer_course	lecturer_id	INT, PK/FK → user_account	Lecturer assigned to the course; part of composite primary key.
lecturer_course	course_id	INT, PK/FK → course	Course taught by the lecturer; part of composite primary key.
lecturer_course	assigned_at	DATETIME, NOT NULL	Timestamp of lecturer-course assignment.
enrolment	enrolment_id	INT, PK, AUTO_INCREMENT	Unique enrolment record identifier.
enrolment	student_id	INT, NOT NULL, FK → user_account	Student enrolled; paired with course_id by a UNIQUE constraint.
enrolment	course_id	INT, NOT NULL, FK → course	Course for this enrolment.
enrolment	enrolled_at	DATETIME, NOT NULL	Enrolment timestamp.
assignment	assignment_id	INT, PK, AUTO_INCREMENT	Unique assignment identifier.
assignment	course_id	INT, NOT NULL, FK → course	Course to which the assignment belongs.
assignment	lecturer_id	INT, NOT NULL, part of composite FK → lecturer_course	Lecturer responsible; composite FK ensures this lecturer is assigned to the course.
assignment	title	VARCHAR(180), NOT NULL	Assignment title.
assignment	instructions	TEXT, nullable	Assignment instructions.
assignment	deadline	DATETIME, NOT NULL	Submission deadline, adjusted by any approved extension.
assignment	max_mark	DECIMAL(7,2), NOT NULL, CHECK > 0	Maximum possible mark; service also validates mark_value ≤ max_mark.
assignment	created_at	DATETIME, NOT NULL	Assignment creation timestamp.
extension_grant	extension_id	INT, PK, AUTO_INCREMENT	Unique extension record identifier.
extension_grant	assignment_id	INT, NOT NULL, FK → assignment	Assignment for which an extension is granted.
extension_grant	student_id	INT, NOT NULL, FK → user_account	Student receiving the extension.
extension_grant	granted_by	INT, NOT NULL, FK → user_account	User who grants the extension; role/course scope checked by service.
extension_grant	revised_deadline	DATETIME, NOT NULL	New effective deadline.
extension_grant	reason	VARCHAR(500), NOT NULL	Reason for the extension.
extension_grant	granted_at	DATETIME, NOT NULL	Timestamp when extension was granted.
submission	submission_id	INT, PK, AUTO_INCREMENT	Unique attempt identifier.
submission	assignment_id	INT, NOT NULL, FK → assignment	Assignment being submitted.
submission	student_id	INT, NOT NULL, FK → user_account	Student submitting the attempt.
submission	submitted_at	DATETIME, NOT NULL	Server-recorded submission timestamp.

submission	file_name	VARCHAR(255), NOT NULL	Validated original/display filename.
submission	file_location	VARCHAR(500), NOT NULL	Application-generated path inside the controlled submission directory.
submission	status	ENUM, NOT NULL	SUBMITTED, LATE, MARKED, MODERATED or RELEASED.
submission	version_no	INT, NOT NULL, default 1, CHECK ≥ 1	Attempt number; unique with assignment_id and student_id.
mark_record	mark_id	INT, PK, AUTO_INCREMENT	Unique mark record identifier.
mark_record	submission_id	INT, NOT NULL, UNIQUE FK → submission	One current mark record at most per submission.
mark_record	lecturer_id	INT, NOT NULL, FK → user_account	Lecturer who entered the mark.
mark_record	mark_value	DECIMAL(7,2), NOT NULL, CHECK ≥ 0	Awarded mark; service validates it does not exceed assignment.max_mark.
mark_record	feedback	TEXT, NOT NULL	Written feedback for the student.
mark_record	moderation_status	ENUM, NOT NULL	PENDING, RETURNED, APPROVED or RELEASED.
mark_record	marked_at	DATETIME, NOT NULL	Timestamp when the mark was recorded.
mark_record	moderated_by	INT, nullable, FK → user_account	HoD or authorised moderator.
mark_record	moderated_at	DATETIME, nullable	Moderation timestamp.
mark_record	released_by	INT, nullable, FK → user_account	User who released the approved result.
mark_record	released_at	DATETIME, nullable	Release timestamp.
audit_log	audit_id	BIGINT, PK, AUTO_INCREMENT	Unique audit event identifier.
audit_log	mark_id	INT, nullable, FK → mark_record	Related mark; nullable to preserve audit history if the mark reference is later unavailable.
audit_log	actor_id	INT, NOT NULL, FK → user_account	User who performed the action.
audit_log	action_type	VARCHAR(40), NOT NULL	Type of mark/moderation/release action.
audit_log	changed_at	DATETIME, NOT NULL	Timestamp of the action.
audit_log	old_value	DECIMAL(7,2), nullable	Previous mark value, where applicable.
audit_log	new_value	DECIMAL(7,2), nullable	New mark value, where applicable.
audit_log	reason	VARCHAR(500), nullable	Reason for the change.
audit_log	details	JSON, nullable	Additional structured audit context.

6.3 MySQL schema

The complete executable MySQL DDL is supplied as Appendix A and as a separate file named 07_Moderated_LMS_MySQL_Schema.sql. It targets MySQL 8.0.16+ and InnoDB. It should be executed first in a development environment, reviewed, and then reverse-engineered in MySQL Workbench to create the editable EER model. The schema is deliberately kept as a separate artifact to make it easier to execute and maintain.

6.4 Constraints and implementation notes

- MySQL InnoDB foreign keys maintain referential integrity between parent and child tables. No SQLite PRAGMA statements are used; foreign-key constraints are declared in the MySQL schema.
- The composite foreign key from assignment (lecturer_id, course_id) to lecturer_course ensures the lecturer-course assignment exists.
- The unique key uq_submission_version prevents duplicate versions for the same student and assignment.
- The service layer must validate role identity, department scope, enrolment, effective deadlines, allowed file types and file size.

- The service layer must reject marks below zero or above assignment.max_mark. Because the maximum resides in another table, this cross-table rule is checked in application logic within a transaction.
- Audit writes must be performed by a controlled application service; restrict direct database access so ordinary users cannot alter or delete audit history.
- Use transactions for mark changes, moderation and release, and parameterised statements for all values.
- Use one consistent timestamp convention; define calendar-day versus working-day late calculations with stakeholders.
- Generate file paths inside a controlled submission directory; never trust an arbitrary path supplied by a user. Define backup and restore procedures.
- Test performance using representative data for approximately 800 students, with indexes on course/deadline, student/status and moderation status.

6.5 Mapping from classes to tables

Class / domain concept	MySQL table(s)	Mapping notes
User / Administrator / Lecturer / Student / HeadOfDepartment	user_account	One account table with role discriminator; role-specific permissions are enforced by services. Department scope is held by department_id.
Department	department	One row per department; referenced by users and courses.
Course	course	Course metadata and department foreign key.
Lecturer-course association	lecturer_course	Junction table; composite primary key prevents duplicate assignment.
Student-course association	enrolment	Junction table with unique student/course pair.
Assignment	assignment	Includes course, assigned lecturer, deadline and maximum mark.
Individual extension	extension_grant	Stores student-specific revised deadline, grantor and reason.
Submission	submission	One row per attempt; version_no preserves resubmission history.
MarkRecord	mark_record	One current mark/feedback record per submission; moderation and release metadata are fields.
AuditEntry / AuditLog	audit_log	One row per auditable action, with actor, time, old/new values and reason.

7. Requirements-to-Design Traceability

The following table connects the principal requirements in the assignment brief to design elements and verification activities. It is a design-level traceability baseline; the Phase 2 requirements specification remains the source of the final requirement identifiers and wording (National University of Lesotho, 2026).

ID	Requirement summary	Design trace	Verification evidence
R-01	Administrator manages users, departments, courses, lecturer assignments and enrolments.	Role access; user_account, department, course, lecturer_course, enrolment.	Authorised admin can create/update records; other roles are denied.
R-02	Lecturer creates assignments with deadlines and maximum marks.	AssignmentService; assignment table; class model.	Create valid assignment; reject missing deadline and non-positive maximum.
R-03	Lecturer grants individual extensions.	ExtensionService; extension_grant table.	Only authorised lecturer can grant extension for assigned course; effective deadline is updated.
R-04	Student views enrolled courses and assignment details.	EnrolmentService; enrolment/course/assignment tables.	Student sees enrolled courses only; unauthorised course query is denied.
R-05	Student submits and resubmits before deadline; late work follows policy.	SubmissionService; state machine; submission table; version unique key.	On-time, late, extended-deadline and duplicate-version tests.
R-06	Lecturer marks submissions and writes feedback.	MarkingService; mark_record table.	Valid mark and non-empty feedback persist; invalid mark is rejected.
R-07	HoD reviews/moderates and releases approved marks.	ModerationService; mark_record; activity and sequence diagrams.	Unapproved mark cannot be released; student cannot view unreleased mark.
R-08	Every mark change is audited with actor, time, old/new values and reason.	AuditService; audit_log; transaction rule.	Audit entry matches each change; failed transaction leaves no partial update.
R-09	Passwords are protected and role-based access is enforced.	Authentication/authorisation services; user_account.password_hash.	No plaintext password stored; role and scope access tests.
R-10	System works offline and supports about 800 students.	Local MySQL deployment; indexes; backup plan.	Offline functional test; load/performance test with representative 800-student data.

8. Verification and Test Design

Tests should be planned from the requirements and executed at the service and database boundaries. The following cases are proposed design tests, not claims that tests have already been executed.

Test ID	Area	Expected result / acceptance condition	Level
T-01	Authentication	Valid password succeeds; invalid password fails; stored value is a salted hash.	Unit / integration
T-02	Role authorisation	Student cannot administer users; lecturer cannot mark an unassigned course; HoD cannot release outside own department.	Service integration
T-03	Assignment validation	Reject max_mark <= 0; reject a lecturer-course combination that is not assigned.	Unit / database integration
T-04	Effective deadline	No extension uses original deadline; approved extension uses revised deadline.	Unit
T-05	Late penalty boundaries	0 late days: 0%; 1 day: 10%; 2 days: 20%; 3 days: 30%; >3 days follows the approved rule.	Unit; policy decision required
T-06	Submission versioning	First submission version is 1; resubmission increments version;	Integration

		duplicate assignment/student/version is rejected.	
T-07	Submission security	Reject invalid file type/size and paths outside controlled storage; preserve previous attempts.	Integration / security
T-08	Mark bounds	Reject negative mark and mark greater than assignment maximum.	Unit / integration
T-09	Moderation and visibility	Student cannot view marked-but-unreleased result; released result becomes visible only after HoD approval.	Integration / acceptance
T-10	Audit atomicity	Every mark change records actor/time/old/new/reason; forced audit failure rolls back mark change.	Integration / transaction
T-11	Correction loop	HoD returns mark; lecturer updates it; new mark change is audited; HoD can re-review.	Acceptance
T-12	Database integrity	Reject invalid foreign keys and duplicate enrolment, lecturer-course assignment or submission version.	Database integration
T-13	Offline operation	Core use cases run without internet when local application and MySQL service are available.	System test
T-14	Performance	Representative dataset for 800 students supports acceptable response times for login, course views, submissions and moderation queue.	Load test; acceptance threshold to be agreed
T-15	Backup and restore	Create a local backup and restore it to a test database; verify records and audit history.	Recovery test

Before testing begins, the team must define measurable response-time targets and agree the accepted behaviour for submissions beyond the three-day late window. These values should not be invented after testing; they must be approved and recorded in the requirements baseline.

9. Design Decisions and Limitations

The design is sufficiently structured to guide implementation, but the following items require stakeholder confirmation at the design gate:

- Whether submissions more than three days late are rejected or accepted with a zero outcome; ensure the SRS, state machine, activity model and tests all use the same decision.
- Whether late days are calendar days or working days, and how the deadline timestamp and time zone are interpreted.
- Supported submission file types, maximum file size and retention policy for submitted files.
- Whether a released mark may be changed; if so, define who authorises it and what additional audit event is required.
- Measurable performance acceptance criteria for the expected 800-student population.
- Password-hashing library, local backup frequency, retention and restore procedure.
- Generate and save the actual MySQL Workbench EER model (.mwb) from the corrected DDL and capture a genuine screenshot for tool evidence.

The corrected schema provides relational integrity and a baseline for implementation. It does not replace application-level validation for role scope, submission deadlines, cross-table mark limits or the late-penalty policy. These rules must be implemented in services and verified by the tests in Section 8.

References

International Organization for Standardization and International Electrotechnical Commission (ISO/IEC/IEEE) (2018) ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering. Geneva: ISO. Available at: https://www.iso.org/standard/72089.html (Accessed: 19 October 2026).

MySQL (n.d.) MySQL 8.0 Reference Manual: FOREIGN KEY Constraints. Available at: https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html (Accessed: 19 October 2026).

MySQL (n.d.) MySQL 8.0 Reference Manual: START TRANSACTION, COMMIT, and ROLLBACK. Available at: https://dev.mysql.com/doc/refman/8.0/en/sql-transactional-statements.html (Accessed: 19 October 2026).

National University of Lesotho, Department of Mathematics and Computer Science (2026) CS4433: Software Engineering — Individual Assignment: Moderated Learning Management System. Maseru: National University of Lesotho.

Object Management Group (2017) OMG Unified Modeling Language (OMG UML), Version 2.5.1. Available at: https://www.omg.org/spec/UML/2.5.1 (Accessed: 19 October 2026).

Sommerville, I. (2016) Software Engineering. 10th edn. Boston: Pearson.

Appendix A: MySQL DDL

The full DDL is supplied as a separate .sql file in the revised package. The code is reproduced below for convenience. Use MySQL 8.0.16+ and run only in a development environment until the schema is reviewed.

```
-- CS4433 Phase 3: Moderated LMS database schema
-- Target: MySQL 8.0.16+ with InnoDB and utf8mb4.
-- Run in a development database first. Then use Database > Reverse Engineer
-- in MySQL Workbench to generate and save the EER model.
CREATE DATABASE IF NOT EXISTS moderated_lms
  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE moderated_lms;

CREATE TABLE department (
  department_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE user_account (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  department_id INT NULL,
  username VARCHAR(80) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  full_name VARCHAR(160) NOT NULL,
  email VARCHAR(160) NULL UNIQUE,
  role ENUM('ADMINISTRATOR','LECTURER','HOD','STUDENT') NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_user_department FOREIGN KEY (department_id)
    REFERENCES department(department_id) ON UPDATE CASCADE ON DELETE SET NULL
) ENGINE=InnoDB;

CREATE TABLE course (
  course_id INT AUTO_INCREMENT PRIMARY KEY,
  department_id INT NOT NULL,
  course_code VARCHAR(30) NOT NULL UNIQUE,
  title VARCHAR(160) NOT NULL,
  description TEXT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT fk_course_department FOREIGN KEY (department_id)
    REFERENCES department(department_id) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE lecturer_course (
  lecturer_id INT NOT NULL,
  course_id INT NOT NULL,
  assigned_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (lecturer_id, course_id),
  CONSTRAINT fk_lc_lecturer FOREIGN KEY (lecturer_id)
```

```
    REFERENCES user_account(user_id) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_lc_course FOREIGN KEY (course_id)
    REFERENCES course(course_id) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE enrolment (
  enrolment_id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  enrolled_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_student_course (student_id, course_id),
  CONSTRAINT fk_enrolment_student FOREIGN KEY (student_id)
    REFERENCES user_account(user_id) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_enrolment_course FOREIGN KEY (course_id)
    REFERENCES course(course_id) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE assignment (
  assignment_id INT AUTO_INCREMENT PRIMARY KEY,
  course_id INT NOT NULL,
  lecturer_id INT NOT NULL,
  title VARCHAR(180) NOT NULL,
  instructions TEXT NULL,
  deadline DATETIME NOT NULL,
  max_mark DECIMAL(7,2) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_assignment_max_mark CHECK (max_mark > 0),
  CONSTRAINT fk_assignment_course FOREIGN KEY (course_id)
    REFERENCES course(course_id) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_assignment_lecturer_course
    FOREIGN KEY (lecturer_id, course_id)
    REFERENCES lecturer_course(lecturer_id, course_id)
    ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE extension_grant (
  extension_id INT AUTO_INCREMENT PRIMARY KEY,
  assignment_id INT NOT NULL,
  student_id INT NOT NULL,
  granted_by INT NOT NULL,
  revised_deadline DATETIME NOT NULL,
  reason VARCHAR(500) NOT NULL,
  granted_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_extension_assignment_student (assignment_id, student_id),
  CONSTRAINT fk_extension_assignment FOREIGN KEY (assignment_id)
    REFERENCES assignment(assignment_id) ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT fk_extension_student FOREIGN KEY (student_id)
    REFERENCES user_account(user_id) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_extension_granted_by FOREIGN KEY (granted_by)
    REFERENCES user_account(user_id) ON UPDATE CASCADE ON DELETE RESTRICT
```

```
) ENGINE=InnoDB;

CREATE TABLE submission (
  submission_id INT AUTO_INCREMENT PRIMARY KEY,
  assignment_id INT NOT NULL,
  student_id INT NOT NULL,
  submitted_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  file_name VARCHAR(255) NOT NULL,
  file_location VARCHAR(500) NOT NULL,
  status ENUM('SUBMITTED','LATE','MARKED','MODERATED','RELEASED') NOT NULL
DEFAULT 'SUBMITTED',
  version_no INT NOT NULL DEFAULT 1,
  CONSTRAINT chk_submission_version CHECK (version_no >= 1),
  UNIQUE KEY uq_submission_version (assignment_id, student_id, version_no),
  KEY ix_submission_assignment_student (assignment_id, student_id),
  CONSTRAINT fk_submission_assignment FOREIGN KEY (assignment_id)
    REFERENCES assignment(assignment_id) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_submission_student FOREIGN KEY (student_id)
    REFERENCES user_account(user_id) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE mark_record (
  mark_id INT AUTO_INCREMENT PRIMARY KEY,
  submission_id INT NOT NULL UNIQUE,
  lecturer_id INT NOT NULL,
  mark_value DECIMAL(7,2) NOT NULL,
  feedback TEXT NOT NULL,
  moderation_status ENUM('PENDING','RETURNED','APPROVED','RELEASED') NOT NULL
DEFAULT 'PENDING',
  marked_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  moderated_by INT NULL,
  moderated_at DATETIME NULL,
  released_by INT NULL,
  released_at DATETIME NULL,
  CONSTRAINT chk_mark_value_nonnegative CHECK (mark_value >= 0),
  CONSTRAINT fk_mark_submission FOREIGN KEY (submission_id)
    REFERENCES submission(submission_id) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_mark_lecturer FOREIGN KEY (lecturer_id)
    REFERENCES user_account(user_id) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_mark_moderator FOREIGN KEY (moderated_by)
    REFERENCES user_account(user_id) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_mark_releaser FOREIGN KEY (released_by)
    REFERENCES user_account(user_id) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE audit_log (
  audit_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  mark_id INT NULL,
  actor_id INT NOT NULL,
  action_type VARCHAR(40) NOT NULL,
```

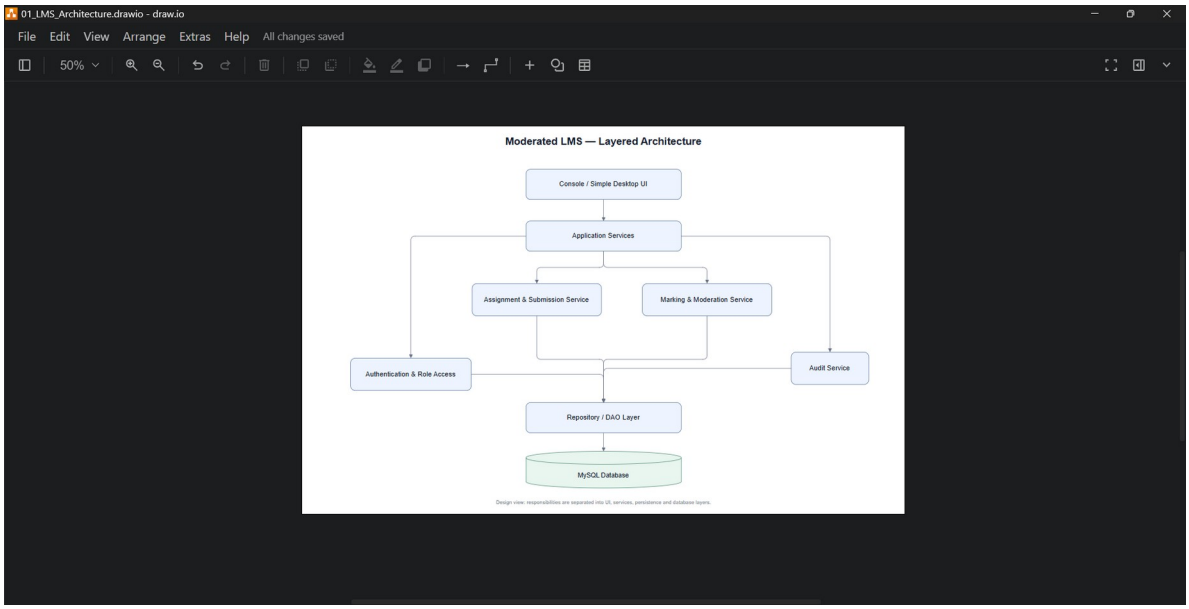
```
  changed_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  old_value DECIMAL(7,2) NULL,
  new_value DECIMAL(7,2) NULL,
  reason VARCHAR(500) NULL,
  details JSON NULL,
  CONSTRAINT fk_audit_mark FOREIGN KEY (mark_id)
    REFERENCES mark_record(mark_id) ON UPDATE CASCADE ON DELETE SET NULL,
  CONSTRAINT fk_audit_actor FOREIGN KEY (actor_id)
    REFERENCES user_account(user_id) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE INDEX ix_assignment_course_deadline ON assignment(course_id, deadline);
CREATE INDEX ix_submission_student_status ON submission(student_id, status);
CREATE INDEX ix_mark_moderation_status ON mark_record(moderation_status);

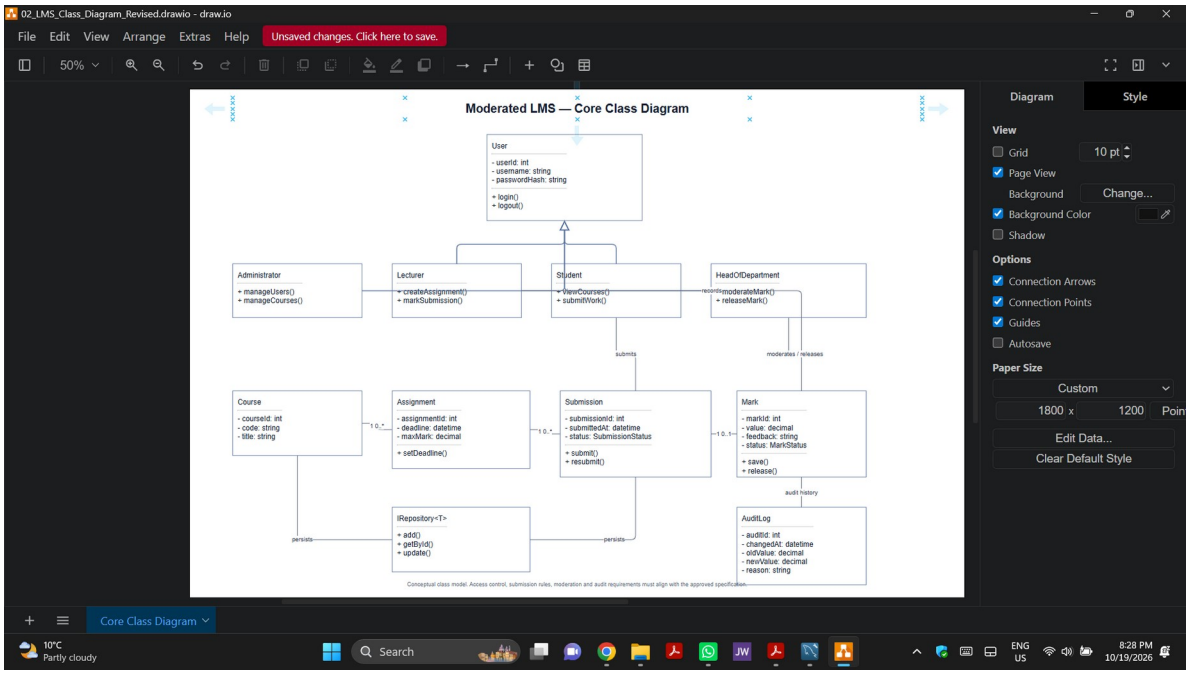
-- Cross-table business rules (e.g., mark_value <= assignment.max_mark,
-- correct role for actor, enrolled student, permitted late window) must be
-- validated by application services within transactions. They cannot be
-- expressed by ordinary single-table CHECK constraints.
```

Appendix B: Diagram Tool Evidence

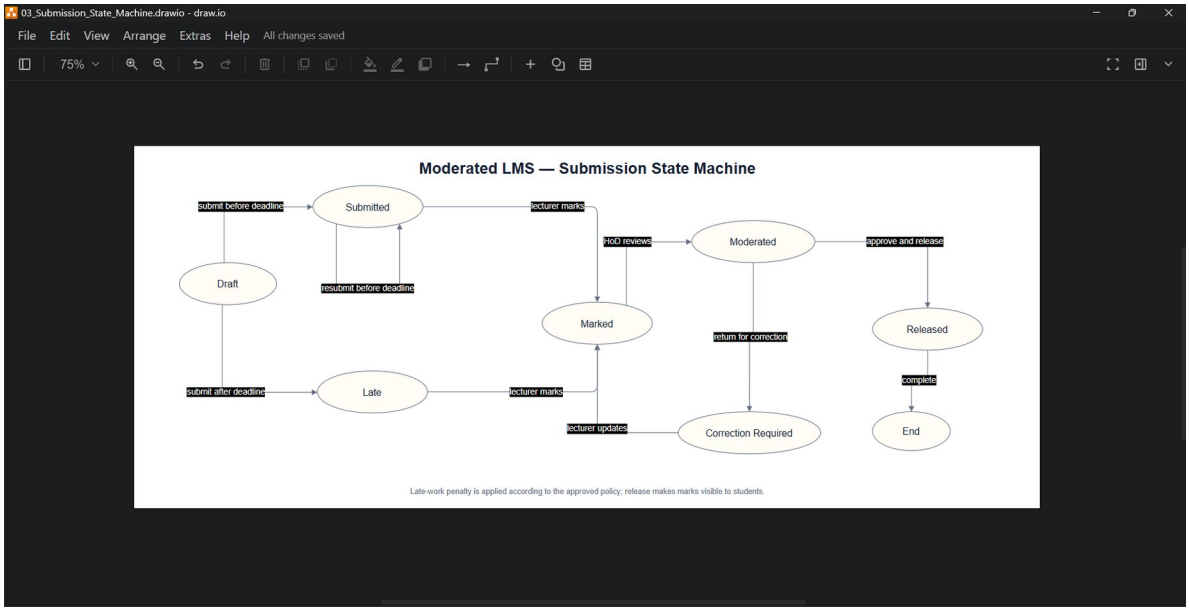
The following are the original screenshots supplied for this report. They show the six UML diagrams open in draw.io. The cropped versions appear in the relevant report sections for readability.



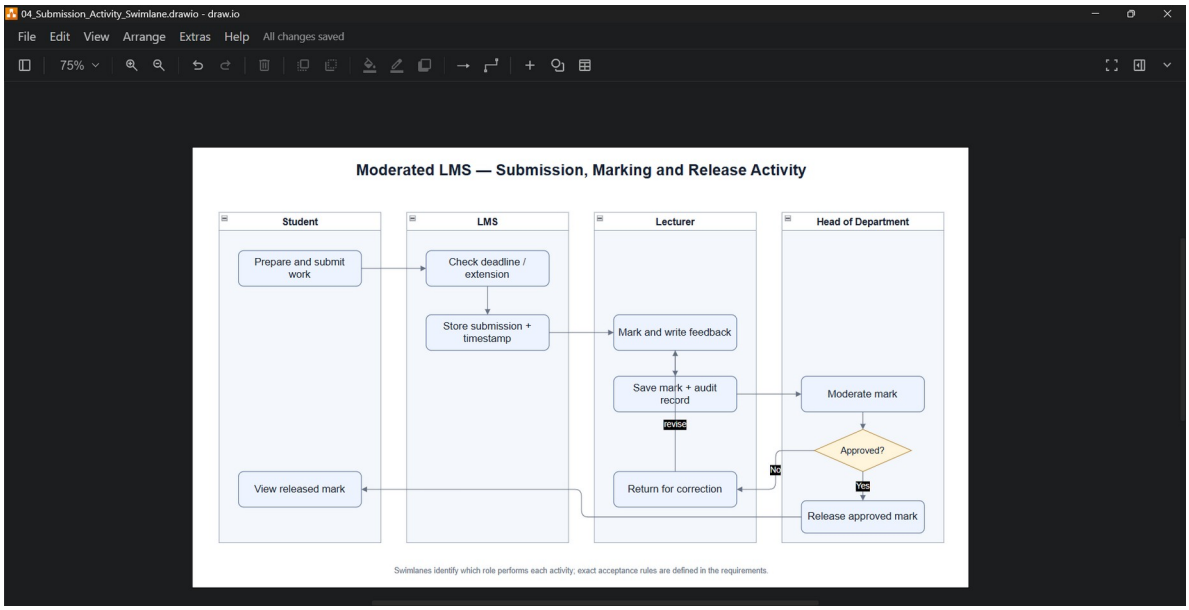
Evidence B1. Original draw.io screenshot for Figure 1.



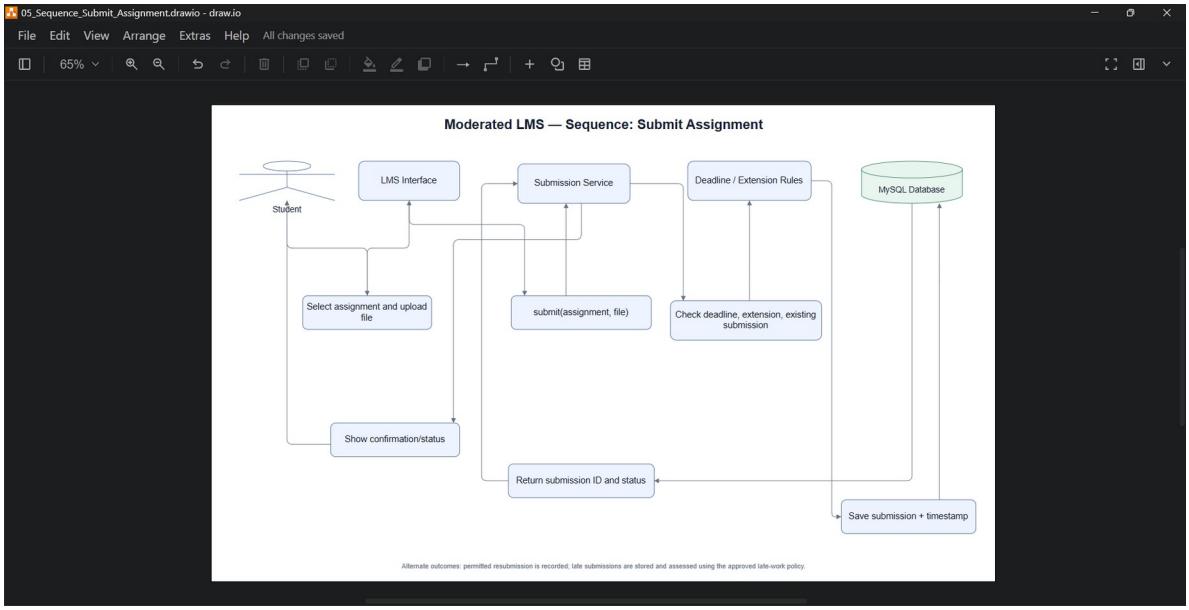
Evidence B2. Original draw.io screenshot for Figure 2.



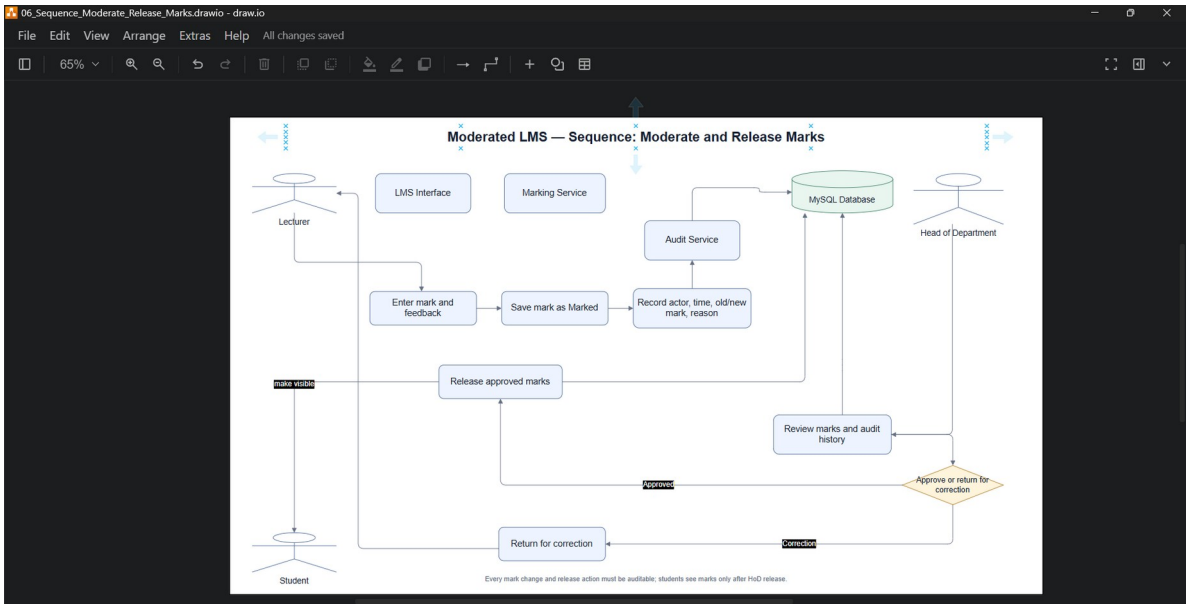
Evidence B3. Original draw.io screenshot for Figure 3.



Evidence B4. Original draw.io screenshot for Figure 4.



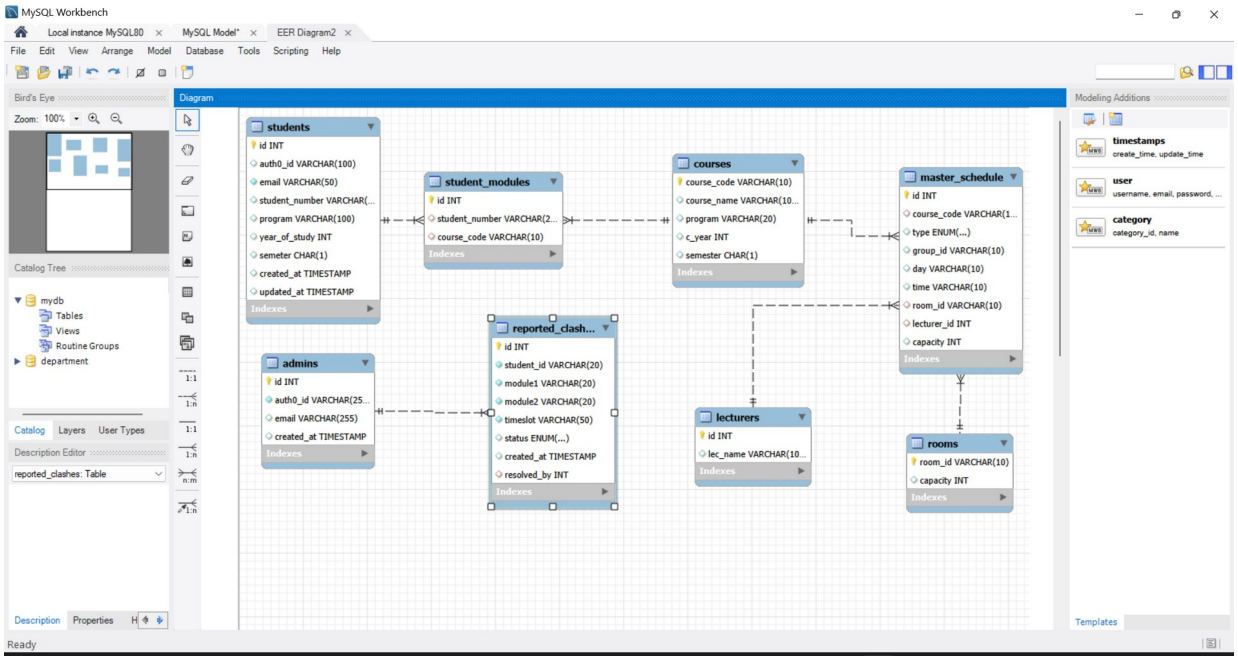
Evidence B5. Original draw.io screenshot for Figure 5.



Evidence B6. Original draw.io screenshot for Figure 6.

Appendix C: MySQL Workbench Tool Evidence

This supplied screenshot confirms that MySQL Workbench was used, but its displayed EER diagram belongs to a different timetable schema. It is not evidence of the Moderated LMS ERD; generate and capture the LMS EER diagram from Appendix A before final submission.



Evidence C1. Supplied MySQL Workbench screenshot (unrelated schema; environment evidence only).